ASSIGNMENT-3

Name: P. Greeshma

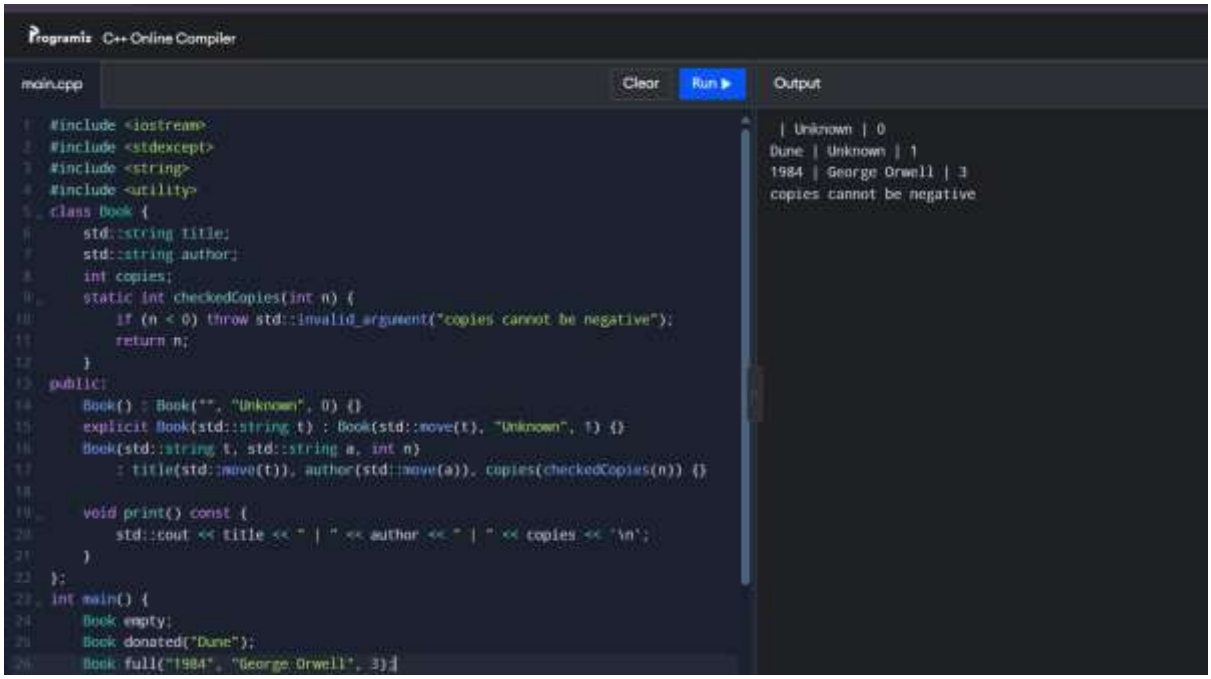
Regno: 192424418

Course Code: DSA0120

Github Link : https://github.com/parvathareddygreeshma-png/DSA0120-Objected-programming-with-c-

Submission date: 09-11-2026

Q1 – Constructors and overloading



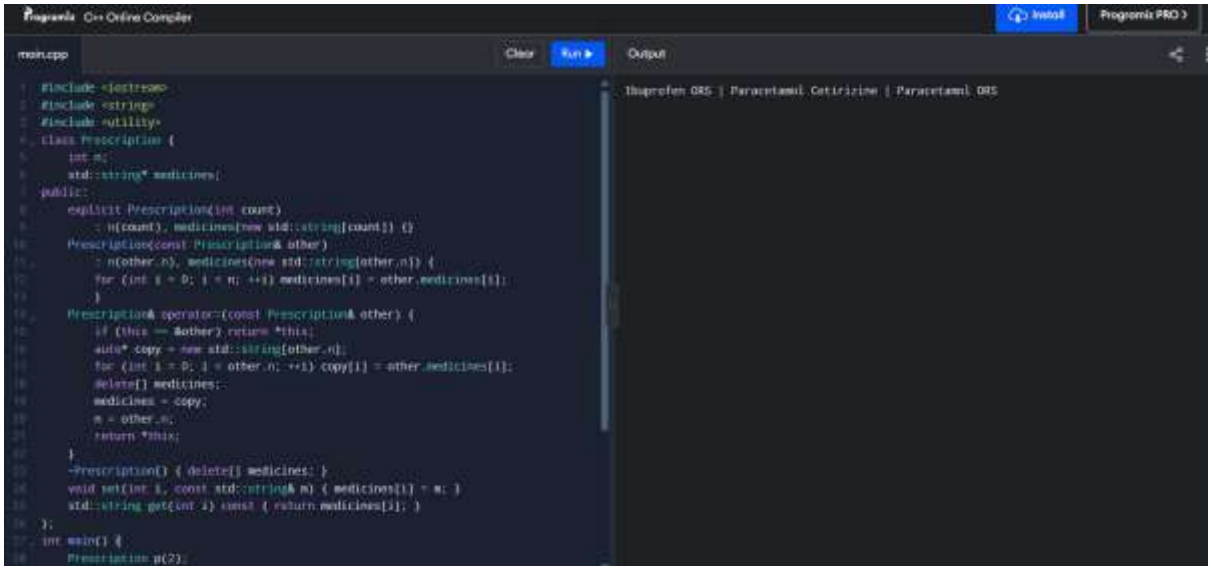
The full constructor validates copies, so no constructor can create a book with a negative count. The other constructors delegate to it. Members initialize in declaration order: title, author, copies.

A compiler supplies a default constructor only when no constructor is declared. If the default constructor is removed, Book books[3]; fails because array elements need a default constructor. Member initializer lists initialize members directly; list order does not change declaration order. Marking the one-argument constructor explicit prevents accidental conversions such as Book b = "Dune";.

Alternative: One constructor with default arguments could replace the overloads. Both designs create objects in O(1) time and space; delegation centralizes validation.

Test: Try a negative copy count and confirm it throws an exception.

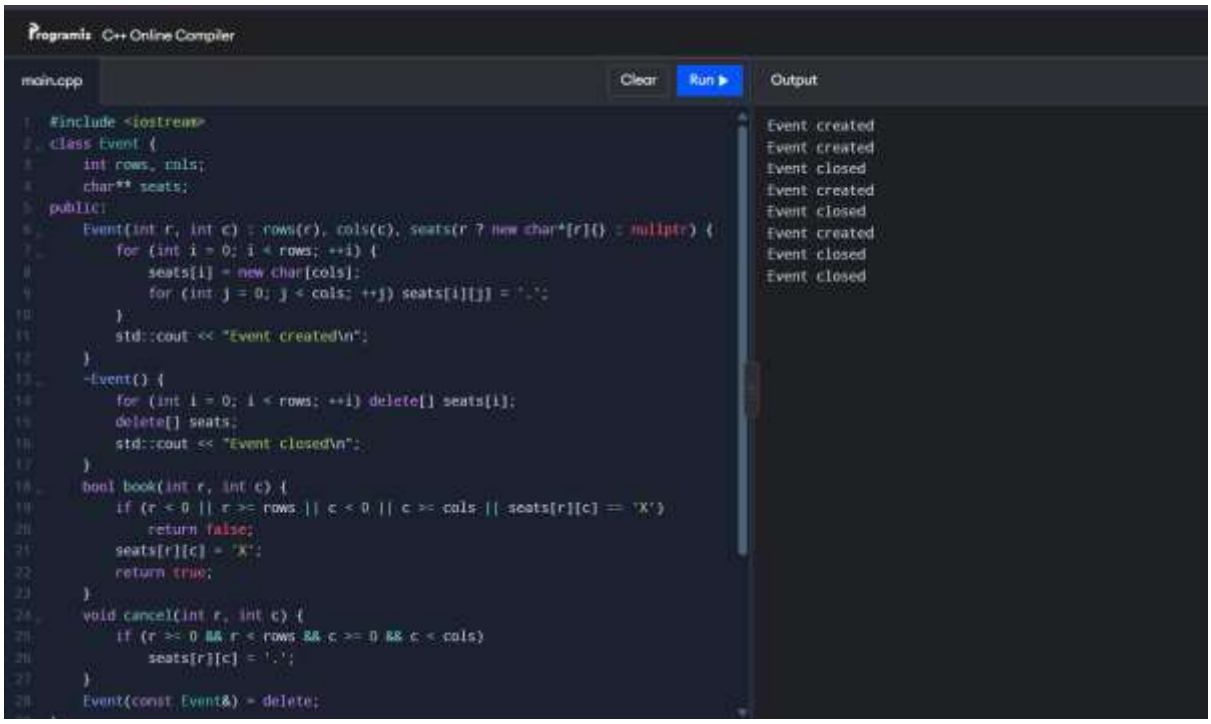
Q2 – Deep copying



A shallow copy copies only the pointer, so objects share the same array and may double-delete it. A deep copy allocates an independent array. The Rule of Three says a class managing memory needs a destructor, copy constructor, and copy assignment operator.

Copying n medicines takes O(n) time and O(n) extra space. std::vector<std::string> can manage copying and cleanup automatically (Rule of Zero). Run with -fsanitize=address to check memory safety.

Q3 – Dynamic constructors and destructors



Local objects are destroyed in reverse construction order when their scope ends. Heap objects are destroyed only by delete; forgetting it leaks memory. Arrays allocated with new[] require delete[]. A destructor takes no arguments and cannot be overloaded. This row-pointer design

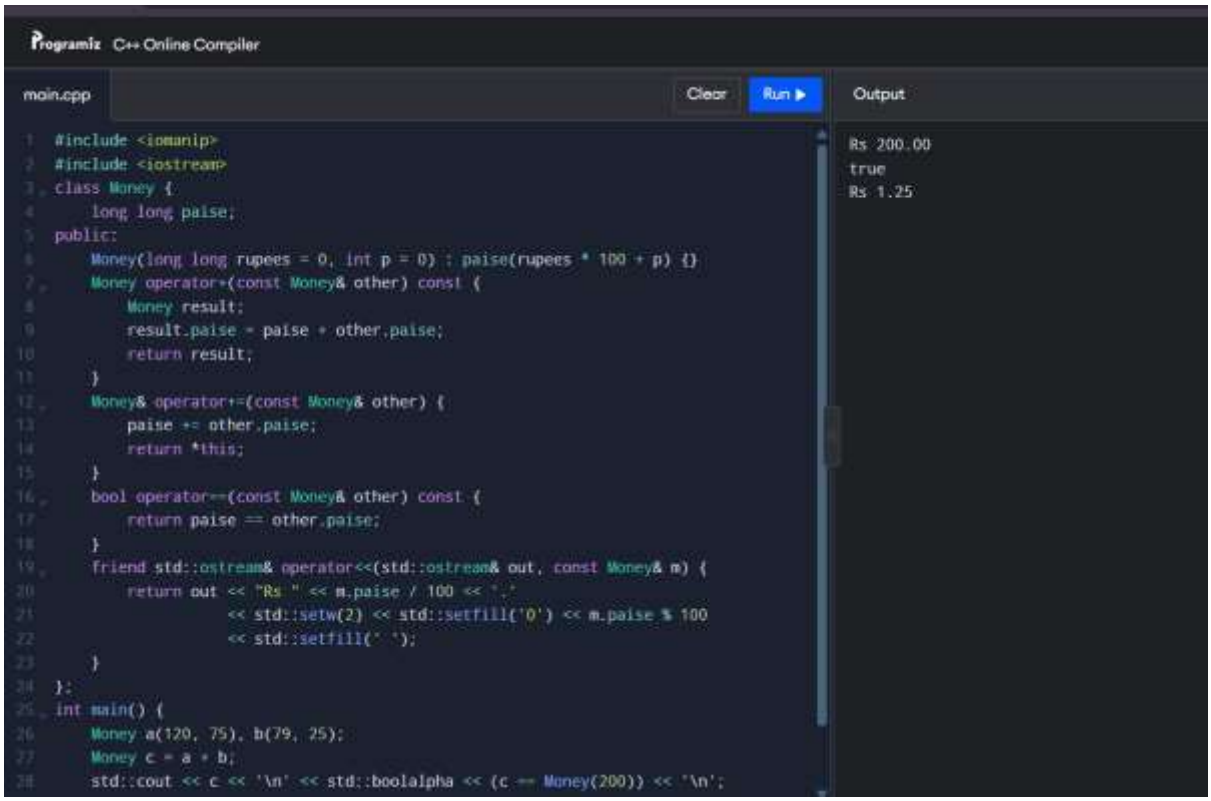
uses rows + 1 allocations; a flat array needs one allocation and indexes seat (r,c) as r * cols + c.

Predict the Logger program's output before running:

+A +B -B copyAc use -Ac +P -P end -A

The parameter to use(Logger) is copied, then destroyed when the call ends. Stack objects are destroyed at scope exit; the heap object is destroyed by delete.

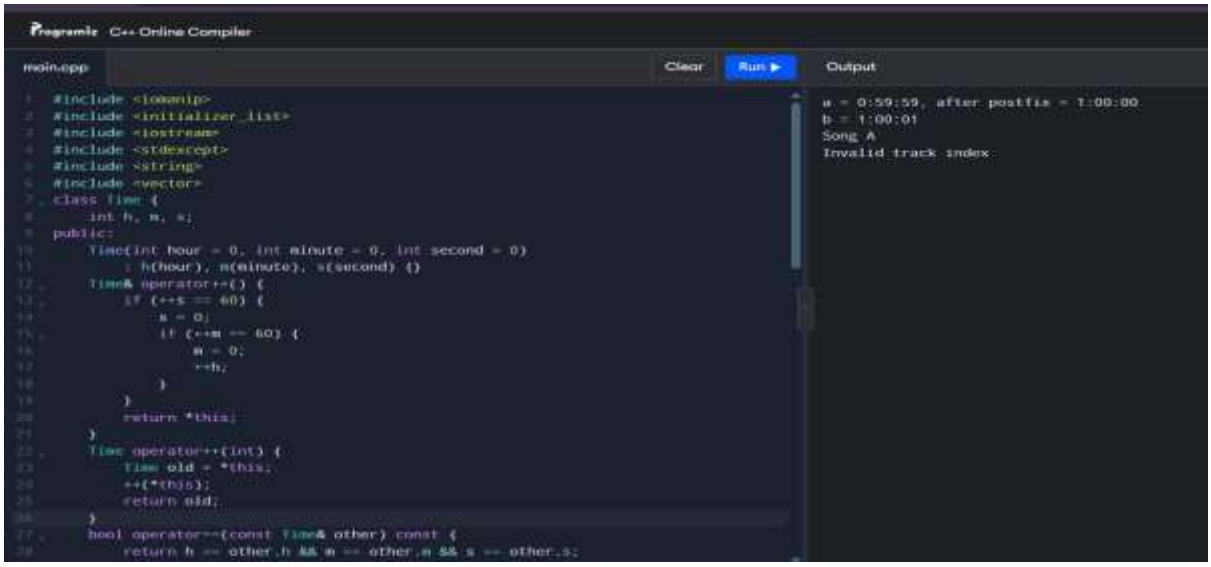
Q4 – Money class



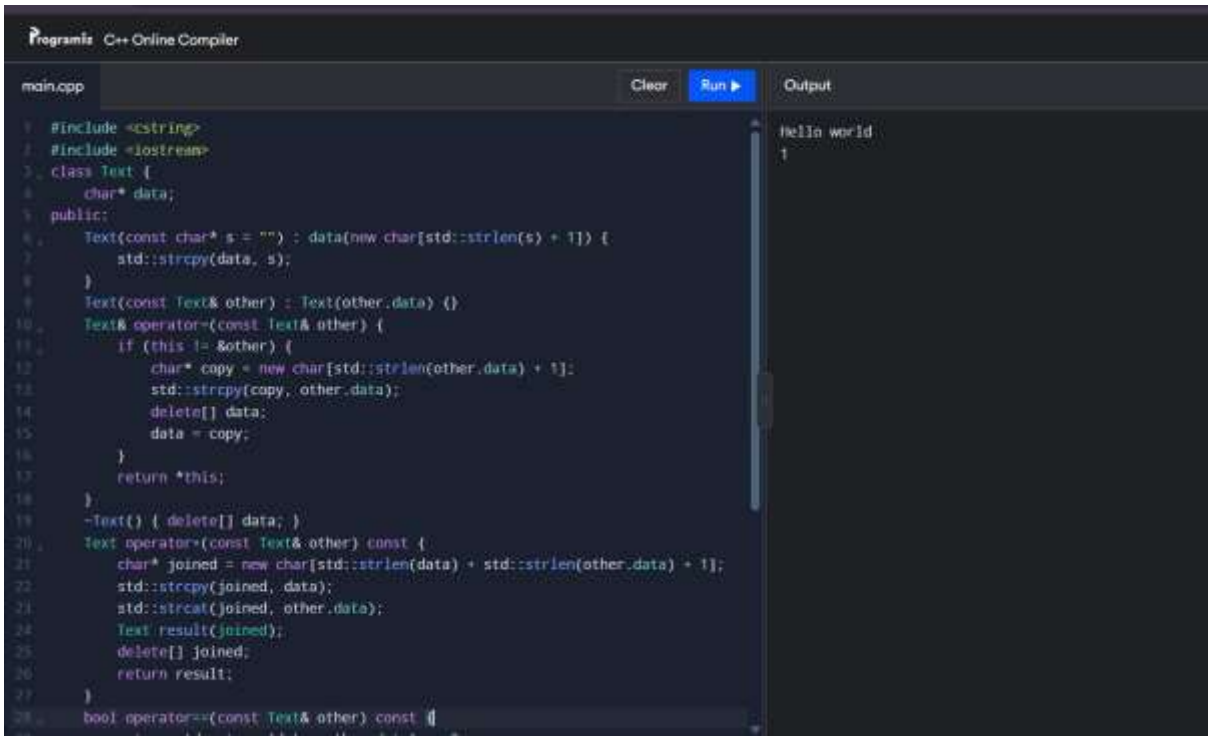
+ returns a new value; += modifies the left side and returns *this by reference. << is a friend non-member so the stream can be its left operand. Operations take O(1) time and space. Money multiplication by another Money value is usually not meaningful.

Q5 – Increment, comparison, and playlist indexing

Predict the provided Counter program: 5 7 7. Postfix returns the old value; prefix increments and returns the updated object. Both take O(1), though postfix makes a copy. Playlist access is O(1); invalid indexes throw an exception. For the required timing evidence, measure 10 million prefix and postfix increments at -O0 and -O2.



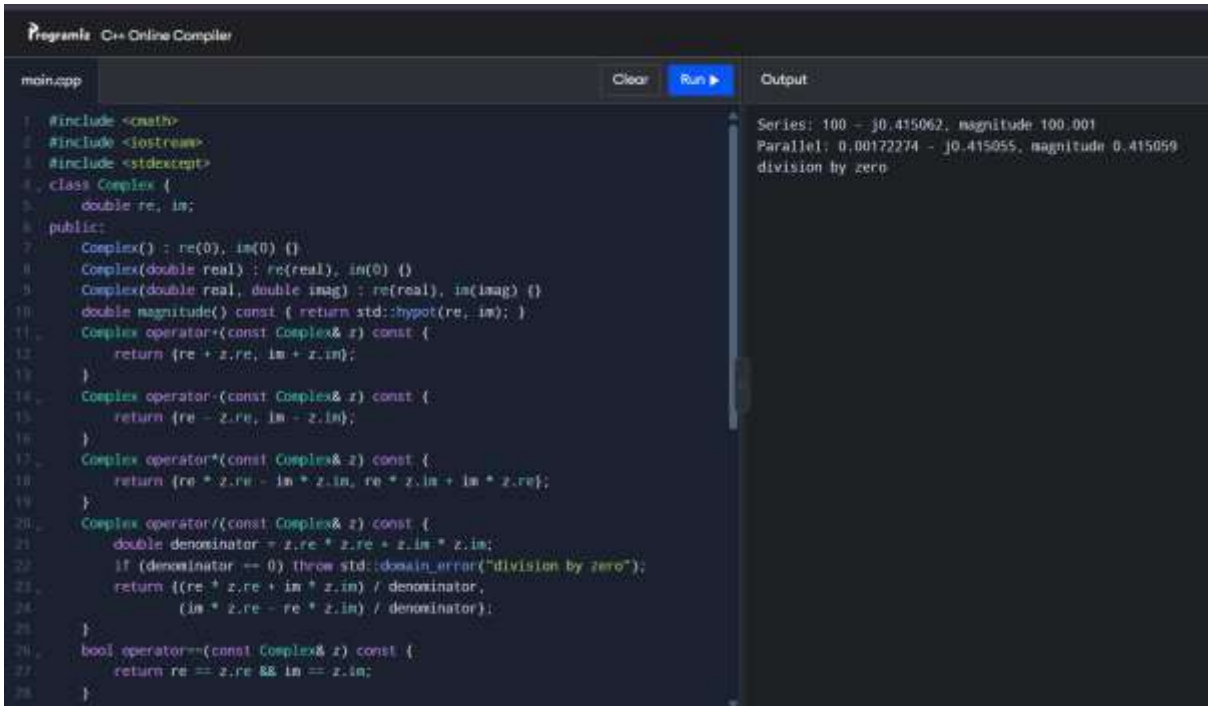
Q6 – Dynamic Text class



operator= must be a member; << is non-member so the stream can be its left operand. Operators such as ::, ., .*, ?:, and sizeof cannot be overloaded. Overloading cannot change precedence or operand count, and one operand must be a user-defined type.

For lengths n1, n2, n3, a + b + c copies about 2n1 + 2n2 + n3 characters across two concatenations. std::string is usually preferable; compare both with 100,000 concatenations and run under AddressSanitizer.

Q7 – Complex numbers for circuit impedance



Formulas: ZL = jωL, ZC = −j/(ωC), series impedances add, and parallel impedance is (Z1 × Z2)/(Z1 + Z2). Each operation takes O(1) time and space. Compare with std::complex<double>; it provides standard complex arithmetic, while your class adds explicit zero-division handling.